

CS 254 Final Report - **Provable Expressivity Limits of Transformer-Based Large Language Models**

March 12, 2026

1 Introduction

Transformer-based large language models have taken the world by storm, with applications in a diverse array of fields ranging from software development, healthcare, education and finance. In this survey, we aim to synthesize the developments in the field of theoretical expressivity analysis of transformers. What can these models, and what cannot, they do in principle?

We approach this question through three complementary lenses. First, we situate transformers within the circuit complexity hierarchy, establishing that constant-depth transformers are precisely characterized by the class TC^0 —a result with immediate, unconditional consequences for what any transformer can compute in a single forward pass, regardless of scale. Second, we examine which formal languages transformers can recognize, revealing surprising gaps even at the lowest tiers of the Chomsky hierarchy. Third, we present impossibility results of direct practical relevance: formal proofs that transformers cannot perform general function composition over large domains, and that hallucination is not an engineering deficiency but a logical inevitability.

Taken together, these results form a rigorous theoretical foundation for understanding the fundamental limits of transformer-based LLMs. Far from diminishing the remarkable empirical success of these models, we argue that such formal characterizations are precisely what is needed to guide the next generation of architectural innovation—and to understand, with mathematical clarity, where the frontier truly lies.

2 Technical Background

Transformer-based large language models build on a deceptively simple idea: instead of processing text sequentially with a recurrent state, they let every token

in a context window attend to every other token via self-attention, then stack many such layers so that rich patterns emerge from massively parallel computation [23]. This architectural shift turns language modeling into the problem of learning to route and mix information through a deep residual network—an ideal setting for both mathematical analysis and large-scale engineering.

At the core lies self-attention, which computes a contextualized representation a_i of the token at position i as a weighted sum of token representations x_j in its (causal) prefix:

$$a_i = \sum_{j \leq i} \alpha_{ij} x_j, \quad (1)$$

$$\alpha_{ij} = \text{softmax}_j (\text{score}(x_i, x_j)). \quad (2)$$

In transformers, each input $x_j \in \mathbb{R}^d$ is linearly projected into a query, key, and value:

$$q_i = x_i W^Q, \quad k_j = x_j W^K, \quad v_j = x_j W^V, \quad (3)$$

and similarity is measured via a scaled dot product:

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}, \quad (4)$$

$$\alpha_{ij} = \text{softmax}_j \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right), \quad (5)$$

$$\text{head}_i = \sum_{j \leq i} \alpha_{ij} v_j. \quad (6)$$

Multi-head attention (MHA) runs this computation A times in parallel with different learned projections (W_c^Q, W_c^K, W_c^V) and concatenates the results:

$$\text{MHA}(x_i, X_{\leq i}) = (\text{head}_i^{(1)} \oplus \dots \oplus \text{head}_i^{(A)}) W^O, \quad (7)$$

so that different heads can specialize to track syntax, long-range coreference, or even algorithmic patterns [23, 7].

Each transformer block wraps this attention mechanism in a carefully engineered residual stack. Given an input $x_i \in \mathbb{R}^d$, a (pre-norm) decoder block applies layer normalization [1], multi-head attention, and a position-wise feedforward network:

$$t_i^{(1)} = \text{LayerNorm}(x_i), \quad (8)$$

$$t_i^{(2)} = \text{MHA}(t_i^{(1)}, \{t_j^{(1)}\}_{j \leq i}), \quad (9)$$

$$t_i^{(3)} = x_i + t_i^{(2)}, \quad (10)$$

$$t_i^{(4)} = \text{LayerNorm}(t_i^{(3)}), \quad (11)$$

$$t_i^{(5)} = \text{FFN}(t_i^{(4)}), \quad (12)$$

$$h_i = t_i^{(3)} + t_i^{(5)}. \quad (13)$$

The feedforward network is a two-layer MLP applied independently at each position:

$$\text{FFN}(z) = \phi(zW_1 + b_1)W_2 + b_2, \quad (14)$$

where ϕ is typically ReLU or GeLU and the hidden width d_{ff} is much larger than d . Layer normalization itself standardizes each token vector and then re-scales and shifts it with learned parameters $\gamma, \beta \in \mathbb{R}^d$:

$$\mu = \frac{1}{d} \sum_{k=1}^d x_k, \quad (15)$$

$$\sigma = \sqrt{\frac{1}{d} \sum_{k=1}^d (x_k - \mu)^2}, \quad (16)$$

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sigma} + \beta. \quad (17)$$

Stacking many such blocks yields a deep residual “stream” in which attention heads move information between positions and feedforward layers nonlinearly transform it [7].

To feed text into this architecture, transformers use learned token embeddings and positional encodings. If $E \in \mathbb{R}^{|V| \times d}$ is the token embedding matrix and $P \in \mathbb{R}^{N \times d}$ a positional embedding matrix for a context of length N , then the input matrix $X \in \mathbb{R}^{N \times d}$ is

$$X_i = E[\text{id}(w_i)] + P_i, \quad (18)$$

optionally replacing P_i with sinusoidal or relative position embeddings [23]. The entire stack is trained as an autoregressive language model: after L layers, the final representation $h_N^{(L)}$ for the last token is passed through an “unembedding” matrix $U = E^\top \in \mathbb{R}^{d \times |V|}$ to obtain logits $u \in \mathbb{R}^{|V|}$:

$$u = h_N^{(L)} U, \quad (19)$$

$$y = \text{softmax}(u), \quad (20)$$

and the model is optimized to maximize the log-probability of the observed next token using cross-entropy loss over massive corpora [3]. Sampling schemes like greedy decoding, top- k , and nucleus (top- p) sampling then turn these probabilities into fluent text generations [13].

Crucially for our later results on expressivity, all of these computations parallelize cleanly. Packing the N token embeddings into a matrix X , a single attention head computes

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \quad (21)$$

$$\text{head} = \text{softmax}\left(\frac{\text{mask}(QK^\top)}{\sqrt{d_k}}\right)V, \quad (22)$$

so every layer can be viewed as a small fixed-depth circuit composed of matrix multiplications, pointwise nonlinearities, and softmax. This perspective, together with empirical scaling laws—e.g., loss L falling as a power law in model size N , data D , and compute C ,

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}, \quad (23)$$

[14], sets the stage for asking rigorous questions about what such constant-depth, highly parallel architectures can and cannot compute, no matter how aggressively we scale them.

Historically, transformers emerged by fusing ideas from sequence-to-sequence attention and memory-augmented models. Encoder–decoder attention—using a soft weighting over input encodings to inform a generative decoder—was developed by [9] for handwriting generation and [2] for machine translation, with related work on neural Turing machines and differentiable memory [10]. This idea was extended to self-attention by dropping the explicit encoder/decoder separation and viewing attention as a way of weighting tokens when passing information up a stack of representations [16, 5, 17]. In parallel, memory networks introduced the terminology of keys, queries, and values for content-addressable read–write memory [22, 24]. These threads culminated in the “Attention Is All You Need” architecture [23] that underlies modern LLMs: simple enough to analyze mathematically, yet powerful enough to drive the most capable language models we have today.

3 Circuit Complexity

What does it truly mean for a transformer to “compute” something? While the previous section framed expressivity in terms of probability distributions over text, a far more surgical tool is available: *circuit complexity*. Instead of asking *what* outputs a model can produce, circuit complexity asks *how hard* the underlying computation is—and places that computation inside a rigorous hierarchy of tractability. The payoff is extraordinary: we obtain *unconditional, architectural upper bounds* on transformer power that hold regardless of model size, training data, or optimization procedure.

3.1 Background: The Circuit Complexity Hierarchy

A Boolean circuit C_n computes a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ using a directed acyclic graph of *gates*. A *circuit family* $\{C_n\}_{n \geq 1}$ solves a problem if each C_n correctly handles all inputs of length n . The two key resources are *depth* (length of the longest input-to-output path, measuring sequential dependence) and *size* (total gate count, measuring overall work).

Let AC^0 denote the class of problems solvable by polynomial-size, *constant-depth* Boolean circuits whose AND and OR gates have *unbounded fan-in*. Let

TC^0 denote the same, but augmented with *threshold gates*, each of which outputs 1 if and only if at least k of its inputs equal 1, for some fixed k .

The strict containment $\text{AC}^0 \subsetneq \text{TC}^0$ is one of the great results of structural complexity [8]: parity cannot be computed in AC^0 yet lives comfortably in TC^0 . Furthermore,

$$\text{AC}^0 \subsetneq \text{TC}^0 \subseteq \text{NC}^1 \subseteq \text{L} \subseteq \text{P},$$

placing both classes at the very *base* of the complexity landscape. Yet even TC^0 is not trivial—integer multiplication, iterated addition, and sorting all reside within it—so membership here already implies significant, practically relevant computational power.

3.2 The Upper Bound: Transformers Live in TC^0

We now arrive at one of the most consequential results in the theory of transformers.

Theorem 1 (Merrill et al., 2025). *A transformer encoder with $O(1)$ layers, $O(1)$ attention heads per layer, and $\text{poly}(n)$ -precision arithmetic, processing a sequence of length n , can be simulated by a uniform family of TC^0 circuits.*

The proof is constructive and worth unpacking because each step reveals *why* a particular transformer operation costs so little in circuit depth. The query, key, and value projections are simple matrix-vector multiplications—constant depth in TC^0 since each output coordinate is a weighted sum of inputs, computable by a single threshold gate layer. The attention scores require exponentiating and normalizing across all n positions; the exponentiation is approximated via iterated squaring (constant depth) and the normalization via iterated addition—both canonical TC^0 operations. Feedforward sublayers with common activations (ReLU, GeLU, etc.) can similarly be realized in constant depth with threshold gates. Chain all $O(1)$ layers together and the total circuit depth remains $O(1)$ —firmly in TC^0 .

The *implications* of Theorem 1 are staggering. Any problem that provably requires more than TC^0 —and there are strong candidates, including problems in NC^1 believed to be strictly outside TC^0 under widely-held complexity hypotheses—is **provably unsolvable by any transformer in a single forward pass**, no matter how many parameters it has or how long it was trained. Scale is irrelevant. Architecture is destiny.

3.3 Hard vs. Soft Attention: A Tale of Two Complexity Classes

The TC^0 upper bound applies to *soft* (i.e. standard softmax) attention. When attention is *hard*—attending exclusively to the position(s) achieving the max-

imum score—the model becomes *strictly weaker*, falling into the smaller class AC^0 .

Theorem 2 (Hao et al., 2022). *Transformers with $O(1)$ layers of hard attention are contained in AC^0 .*

This is a profound separation. AC^0 circuits cannot compute parity (Håstad’s theorem) and, more generally, cannot aggregate information in the *counting* sense that threshold gates enable. Hard attention, by collapsing the rich weighted average over all tokens to a single attended token, loses exactly this aggregation power. The model can *locate* relevant positions but cannot *sum up* evidence across them—and that distinction turns out to be the difference between AC^0 and TC^0 .

Chiang et al. [6] sharpen both boundaries considerably. Rather than establishing upper bounds alone, they prove *matching lower bounds*: specific TC^0 problems (e.g. iterated addition over token values) that *can* be implemented by constant-layer soft-attention transformers. This renders the containment essentially *tight* for soft attention, transforming a one-sided upper bound into a nearly complete characterization:

Hard-attention Transformers $\approx \text{AC}^0$ Soft-attention Transformers $\approx \text{TC}^0$.

3.4 Practical Consequences and the Bridge to Applications

Abstract containment results immediately license a list of concrete, practically relevant limitations. Since TC^0 requires constant depth, transformers *cannot emulate sequential computation of super-constant depth in a single forward pass*—every token must be processed “in parallel,” with only $O(1)$ rounds of global communication between positions. This rules out, in principle, algorithms whose inherent sequential depth grows with input length.

Chen et al. [4] drive this point home empirically and theoretically, connecting the circuit complexity framework to concrete reasoning benchmarks. They show that several arithmetic and multi-step logical reasoning tasks, which require computation depth growing with the input, cannot be solved by bounded-depth transformers—providing a direct, formal explanation for the notorious difficulty of arithmetic reasoning in LLMs. Crucially, their analysis demonstrates that the culprit is not insufficient scale but *architectural depth limitation*: a provable, permanent feature of the transformer design.

Taken together, this body of work delivers a clean, satisfying answer to the question “how powerful is a transformer?”—not in the fuzzy empirical sense, but in the rigorous, unconditional sense that complexity theory demands. The transformer is a constant-depth threshold circuit. Nothing more, nothing less.

4 Formal Language Recognition

Circuit complexity places transformers inside a computational box. Now we ask a more linguistically natural question: which *languages* can a transformer recognize? Formal language theory, the classical mathematical study of sets of strings and the automata that accept them, provides the vocabulary. The results are both illuminating and—once again—sobering: there are remarkably simple languages that transformers, in their standard form, provably cannot handle.

4.1 The Chomsky Hierarchy: A Quick Primer

Recall that formal languages are partitioned into the classical *Chomsky hierarchy*:

$$\mathcal{L}_{\text{Reg}} \subsetneq \mathcal{L}_{\text{CF}} \subsetneq \mathcal{L}_{\text{CS}} \subsetneq \mathcal{L}_{\text{RE}},$$

corresponding, respectively, to finite automata (FSAs), pushdown automata, linear-bounded automata, and Turing machines. One of the most natural questions one can ask about a new computational model is: *where does it sit in this hierarchy?* For transformers, the answer is subtle, attention-variant-dependent, and frankly surprising.

Strobl et al. [21] provide the most comprehensive map of this territory to date—a landmark survey synthesizing a decade of results into a unified framework. Their findings reveal that transformer expressivity does not cleanly correspond to any single level of the Chomsky hierarchy; instead, different attention mechanisms and architectural choices yield a patchwork of capabilities that cut *across* classical boundaries in unexpected ways.

4.2 Hard Attention and the Poverty of Sequential Memory

Consider the DYCK_k language: the set of all well-matched parenthesis strings over k bracket types. For example, with $k = 2$:

$$([] ())$$

is in DYCK_2 , while $([])$ is not. DYCK_k is the canonical context-free language; it captures the nested, recursive structure underlying virtually all natural and formal languages, from arithmetic expressions to program syntax to relative clauses. If a model cannot recognize DYCK_k , it lacks a fundamental building block of linguistic competence.

Theorem 3 (Hahn, 2020). *For any fixed transformer with hard (saturated) attention and a bounded number of layers, the model’s acceptance probability on PARITY and DYCK_k ($k \geq 2$) converges to chance as sequence length $n \rightarrow \infty$. Concretely, no such transformer can achieve error $< 1/4$ on PARITY/DYCK₂ for all sufficiently long strings.*

The proof relies on a beautiful argument: hard attention heads can each attend to *at most one position* at each step, so the information flow from distant positions to the current one is severely bottlenecked. By bounding the mutual information between the model’s output and arbitrarily long suffixes of the input, Hahn shows that the transformer’s output distribution cannot differentiate between exponentially many equivalence classes that DYCK_k does distinguish. In other words, the model simply runs out of “channels” through which long-range dependency information can flow.

To be maximally clear about what this means: **DYCK_k is a context-free language; PARITY is regular.** Hard-attention transformers provably fail on languages as simple as regular ones. The Chomsky hierarchy, developed to classify the expressive power of abstract automata, places these within its lowest non-trivial tier—and yet a hard-attention transformer cannot reach them. This is not a training failure. It is a *fundamental architectural limitation*.

4.3 Soft Attention: More Power, Still Bounded

Does switching to soft attention rescue the situation? Partially, and in an interesting way. Soft-attention transformers can aggregate information across all positions simultaneously, which qualitatively changes the picture.

Theorem 4 (Strobl et al., 2024). *Soft-attention transformers with sufficient precision can recognize all regular languages in TC^0 . However, their ability to recognize languages beyond the regular tier is severely constrained by depth and precision.*

The key insight is that the weighted average in soft attention acts as a global summation over token embeddings—and with the right weight matrices, this can simulate the state transitions of a finite automaton. But as soon as the target language requires *unbounded stack depth* (the hallmark of context-free languages), soft attention cannot provide the requisite “counter” or “stack” memory: the number of distinct internal states it can maintain is bounded by the embedding dimension, which is fixed at training time and does not grow with the input.

Hao et al. [12] formalize this through the circuit complexity lens: since hard-attention transformers are in AC^0 , and since PARITY is known to require TC^0 (i.e. it is not in AC^0), the failure of hard-attention models on PARITY is not an

accident—it is the circuit lower bound in disguise. This elegant duality between formal language recognition and circuit complexity is one of the most satisfying structural results in the field: the two perspectives are not merely analogous but *formally equivalent* for constant-depth transformers.

4.4 A Unified Picture

Putting everything together, the formal language recognition landscape for transformers looks roughly as follows:

Language Class	Hard Attention	Soft Attention
Finite languages	✓	✓
Bounded-length regular	✓	✓
All regular	× (Theorem 3)	✓ (Theorem 4)
Context-free (DYCK_k)	× (Theorem 3)	× (precision-bounded)
Context-sensitive	×	×

Strobl et al. [21] rigorously establish and extend this table across a range of transformer variants—including *unique-hard attention*, *average-hard attention*, and *arbitrary-precision soft attention*—revealing that the differences between these variants are not mere implementation details but genuine differences in computational class. The survey stands as the definitive reference for understanding where any given transformer variant falls in the formal language hierarchy.

4.5 Why These Results Are More Than Academic

One might be tempted to dismiss DYCK_k recognition as a toy problem irrelevant to the real world. Resist that temptation. DYCK_k is the *formal backbone* of syntax: parse trees in natural language are trees, and trees are intimately related to balanced bracket structures. If a transformer cannot reliably recognize DYCK_2 at long lengths, it means the model’s ability to track nested syntactic dependencies—the very structure of hierarchical language—degrades as utterances grow.

Indeed, Hahn’s result [11] provides a formal upper bound on how well a model can track long-range syntactic agreement, co-reference, and other phenomena that require “remembering” an unbounded amount of structural context. Taken together with the compositionality result discussed in Section ??, a coherent picture emerges: transformers face provable, architectural barriers precisely at the points where language is hardest—deeply nested structure, long-range composition, and open-ended symbolic recursion. These are not bugs to be fixed with more data. They are theorems.

5 Impossibility Results

While theoretically interesting, discussions of transformers as circuits or recognizers of formal languages arguably has limited practical use. After all, transformers were not developed to be formal models of computation but instead to model the nebulous, real-world process of natural language. What has been discussed thus far then may not hold much bearing on the ability of transformers to achieve their practical goals of producing realistic text, engaging in complex conversations with users, or assisting in tasks like summarization and retrieval.

However, it is possible to formally prove limitations of transformers that carry serious implications for tasks of practical, everyday interest. Below, we give a (somewhat informal, intuition-focused) overview of two formal proofs dealing with two desired properties of transformers that are extremely significant and relevant to the real-world use of LLMs.

5.1 Compositionality

In this section, we will prove that transformers cannot solve function composition problems for functions whose domains are sufficiently large.

First, we will introduce the notion of a function composition problem and motivate why being able to solve it is critical for a transformer.

Consider the following series of sentences:

1. London is in the UK.
2. Alan Turing was born in London.
3. In which country was Turing born?

To answer the presented question, one must employ function composition in a semantic sense: you must map the person “Turing” to the city “London” through a *born-in* semantic function. Then, you must map the city “London” to the country “UK” through a *country-of* semantic function.

Clearly, this kind of semantic composition is a ubiquitous part of language use. Beyond composing facts together, composition is involved in other linguistic tasks such as detangling what pronouns or indexicals (e.g. “this” or “that”) are referencing. As a result, we would certainly want any language model to be able to execute it. How do we go about proving that transformers cannot?

The proof relies on a lemma which in turn relies on key findings from Communication Complexity, a subfield of Complexity Theory dealing with different agents exchanging bits of information to execute a computation. For the sake of simplicity and focusing on the intuition behind the proof, we state the lemma and a rough justification for it without getting lost in the technical Communication Complexity details.

Suppose we have a function composition problem where the domain size of all inputs is n . Also suppose we have three computational agents Faye, Grace, and Xavier. Faye knows a function f and Grace a function g , both from $[n]$ to $[n]$. Xavier is assigned a number $x \in [n]$ which all agents know. Faye and Grace can each communicate to Xavier and no other communication is allowed.

Lemma: *If fewer than $n \log n$ bits are communicated by Faye to Xavier, then Xavier cannot know the value $f(g(x))$ with 100% certainty.*

Intuition: Intuitively, Xavier must first obtain $g(x)$. To do so, Grace communicates $g(x)$ to Xavier (recall that Grace also knows the value of x). Now, Xavier must apply f to $g(x)$. The problem is that Xavier knows nothing about f .

There are n^n possible functions f . Faye must communicate the nature of f to Xavier for Xavier to calculate $f(g(x))$. To describe f completely requires at least $\log_2(n^n) = n \log n$ bits to be sent. If less bits are used, say some amount $B < n \log n$, then Faye only has 2^B possible bitstrings she can send. $2^B < n^n$, meaning that by the pigeonhole principle, some length B bitstring must map to two distinct $[n]$ to $[n]$ functions f, f' . Since these are distinct, they must disagree on some input z , that is $f(z) \neq f'(z)$. If $g(x) = z$, which is entirely possible, then Xavier has no way of knowing which function to apply and therefore how to obtain $f(g(x))$.

Theorem 5. *No H -headed self-attention layer $\mathcal{L}$ of embedding dimension d and computation precision p can solve the function composition problem if $H(d+1)p < n \log n$. [20]*

Proof sketch: Suppose for the sake of contradiction such a head exists and that we have agents Faye, Grace, and Xavier as in the lemma. These agents can compute $f(g(x))$ using $\mathcal{L}$ if Faye supplies the tokens associated with f 's description (e.g. “ $f(0) = 3, f(1) = \dots$ ”), Grace supplies the tokens associated with g 's description, and Xavier supplies the tokens associated with the query (e.g. “what is $f(g(x))$?”). These are all supplied to each attention head.

For the layer to compute the task successfully, it must output the ground-truth token. By the definition of attention, each head within the layer must output some specific token y_N that, when combined, will lead to outputting the ground-truth token.

For each attention head, by the definition of attention we have

$$y_N = \frac{\sum_{j=1}^N r_{t,j} V x_j}{\sum_{j=1}^N r_{t,j}}, \text{ where } r_{t,j} = \exp(x_t^T Q^T K x_j)$$

That is, at the end of the day, at time step N the head output results from a complex mathematical term realizing self-attention operations (attention scoring via query-key similarity, deriving value vectors from tokens, etc.) applied to the tokens from Faye, Grace, and Xavier. Thus, we can rewrite this as

$$y_N = \frac{F + G + X}{F' + G' + X'}$$

where F, F' is essentially a placeholder term representing all the attention terms derived from Faye’s tokens, and similarly for G, G' for Grace and X, X' for Xavier.

But now this formulation lets us leverage the allowed communication between agents. Namely, Faye can compute F, F' herself, Grace can compute G, G' herself, then they can send their results to Xavier who computes X, X' then finally the solution y_N from all the information he now has.

For a single head, Faye sends dp bits for F (as it is a weighted combination of d -dimensional value vectors) and p bits for F' (as it is a combination of the attention weights for Faye’s tokens). Across the H heads, we then see that Xavier is able to compute $f(g(x))$ correctly with only $H(d + 1)p$ bits of communication from Faye, which is less than $n \log n$ by assumption. This contradicts the lemma. Therefore, no such attention layer $\mathcal{L}$ can exist. $\square$

5.2 Hallucination

We will now give an overview of a proof that hallucination is a logical inevitability for any LLM. To do so, we first must formalize the notion of an LLM and what it means to hallucinate.

Denote an LLM as h . We can train this model over i stages of time, producing models $h^{[1]}, h^{[2]}, \dots, h^{[i]}$, each being a state of the model after some amount of training.

Define a formal world where we have some ground-truth function f and a set of all possible strings $\mathcal{S}$ such that $f(s)$ for any string $s \in \mathcal{S}$ always produces the correct output for s , whatever that means for our choice of task.

Under this formalization, an LLM h hallucinates with respect to a ground-truth function f if $\forall i \in \mathbb{N}. \exists s \in \mathcal{S}. h^{[i]}(s) \neq f(s)$. That is, we can never train h to always match the ground-truth function.

Note that any formal world is a subset of the real world, as all formal worlds can be viewed as more restrictive, well-defined formalizations of real-world natural language tasks.

Table 1: Illustration of diagonalization for the proof of Theorem 1. Each row represents a particular LLM state. Each column represents a string s_i with which the LLM is tested. Blue cells are the outputs used to construct f in Eq. (1)

LLMs	Test Samples				
	s_0	s_1	s_2	s_3	$\dots$
$\hat{h}_0$	$\hat{h}_0(s_0)$	$\hat{h}_0(s_1)$	$\hat{h}_0(s_2)$	$\hat{h}_0(s_3)$	$\dots$
$\hat{h}_1$	$\hat{h}_1(s_0)$	$\hat{h}_1(s_1)$	$\hat{h}_1(s_2)$	$\hat{h}_1(s_3)$	$\dots$
$\hat{h}_2$	$\hat{h}_2(s_0)$	$\hat{h}_2(s_1)$	$\hat{h}_2(s_2)$	$\hat{h}_2(s_3)$	$\dots$
$\hat{h}_3$	$\hat{h}_3(s_0)$	$\hat{h}_3(s_1)$	$\hat{h}_3(s_2)$	$\hat{h}_3(s_3)$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$
f	$\Delta(\hat{h}_0(s_0))$	$\Delta(\hat{h}_1(s_1))$	$\Delta(\hat{h}_2(s_2))$	$\Delta(\hat{h}_3(s_3))$	$\dots$

Figure 1: A diagram of the diagonalization from the original paper

Theorem 6. *For all computably enumerable sets of LLMs $\{h_0, h_1, \dots\}$, there exists a computable ground truth function f such that all $h_i^{[j]}$, $i, j \in \mathbb{N}$ will hallucinate. [25]*

Proof sketch: At its core, this is a standard diagonalization proof. We can take all possible $h_i^{[j]}$ derivable from training the LLMs in our set and re-enumerate them into a set $\{\hat{h}_0, \hat{h}_1, \dots\}$. We can then run each on all possible strings $s_0, s_1, \dots$ and create our f by ensuring that it disagrees with each $\hat{h}_k$ on s_k . Thus, any $h_i^{[j]}$ maps to some $\hat{h}_k$ such that f disagrees with $\hat{h}_k$ on some input, meaning that $h_i^{[j]}$ hallucinates.

□

From this proof, it's easy to extend to more dramatic conclusions.

First, we can first modify f so that for any given $\hat{h}_i$, f disagrees on infinitely many inputs. To do this, define $f(s_i) = \Delta(\{\hat{h}_j(s_i) \mid j \leq i\})$, $\forall i \in \mathbb{N}$ where $\Delta(\{\hat{h}_j(s_i) \mid j \leq i\})$ returns a string different from all strings in $\{\hat{h}_j(s_i) \mid j \leq i\}$. Now by construction, $\hat{h}_j$ will disagree with f and thereby hallucinate on $s_j, s_{j+1}, \dots$.

Second, since any individual computable LLM forms a set $\{h\}$, from the above conclusions we see that any singular LLM must hallucinate on infinitely many inputs.

6 Transformers in Practice

Function composition seems like an essential, basic feature of language. Hallucinations are one of the most bothersome issues about LLMs and seem unintelligent and unlike how humans typically behave when not knowing an answer. If

we can formally prove that transformers are unable to do function composition on any meaningfully large domains and that transformers will always suffer from hallucinations, why do they remain so popular? Additionally, how do we resolve these formal proofs with the reality that transformers have appeared to become significantly better over time at many of the issues that once plagued them?

To take a more philosophical turn, the question is one of *theory vs practice*. While we can formally prove limitations of the transformer architecture, advancements in fields like training and inference techniques mitigate these limitations, potentially to such an extent that they become irrelevant.

Theoretical work has shown that, despite the common idea that scaling can resolve any issue, even arbitrarily scaling up LLMs cannot resolve certain fundamental theoretical issues. For instance, the learning theory framework of PAC learning bounds shows the difficulty for a finite model to compress increasingly large corpora. [19] And yet, scaling up these models has undeniably led to drastic quantitative and qualitative improvements, especially in areas that provably can never be completely addressed, from retrieval to reasoning.

Revisiting the previous two impossibility results, we see a similar pattern. While complete function composition may be theoretically impossible, in practice models have gotten to the point where the issue seems to not matter anymore. Prompt any current model with the composition examples given in the paper and it will trivially solve them, despite models from just a few years ago failing on them. And while there may be certain far-flung edge cases where current models still fail, those may not matter if the model is correct on 99.9% of cases. Additionally, techniques like Retrieval Augmented Generation or RL post-training drastically reduce hallucination rates (for more, see this interesting paper on how LLMs can identify their own lack of knowledge [15]).

In general, there is a critical question of if the mathematical formality that LLMs have certain fundamental issues is really a practical issue. While such issues may provably always be present, it seems more and more possible to mitigate them so drastically that they reach irrelevancy.

7 Conclusion

In our formal exploration of transformers, we first started with a formalization of transformers as circuits, their place within the circuit complexity hierarchy, and the upper bounds on power that creates. From this, we see what formal languages transformers can and cannot recognize, and how their inability to recognize certain formal languages has serious practical consequences on the real-world use of transformers.

We then moved onto formal proofs for two significant impossibility results, that transformers cannot do function composition for sufficiently large domains and that infinite hallucinations are inevitable for any transformer. We then discussed

the distinction between ML theory and how these systems have rapidly improved in practice.

Overall, we believe that formal results such as the ones we have surveyed should be used not as attacks against the use transformers but as ways to understand and guide their development better. It would make no sense to dedicate exhaustive resources towards trying to get rid of mathematical impossibilities. Instead, it seems that fundamental architectural flaws of transformers need to be circumvented in other ways (and this already appears to be what is being done by the field). Undeniably, the transformer has reshaped the entire field of AI with its power and expressivity across many tasks. But this makes knowing its fundamental limits even more important.

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. “Layer Normalization”. In: *NeurIPS Deep Learning Symposium*. 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *International Conference on Learning Representations (ICLR)*. 2015.
- [3] Tom B. Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.
- [4] Xiang Chen et al. “Circuit Complexity Bounds for Transformer-Based Reasoning”. In: (2025).
- [5] Jianpeng Cheng, Li Dong, and Mirella Lapata. “Long Short-Term Memory Networks for Machine Reading”. In: *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2016.
- [6] David Chiang et al. “Tighter Bounds on the Expressivity of Transformer Encoders”. In: 2023.
- [7] Nelson Elhage, Neel Nanda, Catherine Olsson, et al. “A Mathematical Framework for Transformer Circuits”. In: *Transformer Circuits Thread* (2021). White paper.
- [8] Merrick Furst, James B. Saxe, and Michael Sipser. “Parity, Circuits, and the Polynomial-Time Hierarchy”. In: *Mathematical Systems Theory*. Vol. 17. 1. Springer, 1984, pp. 13–27.
- [9] Alex Graves. “Generating Sequences with Recurrent Neural Networks”. In: *arXiv preprint arXiv:1308.0850* (2013).
- [10] Alex Graves, Greg Wayne, and Ivo Danihelka. “Neural Turing Machines”. In: *arXiv preprint arXiv:1410.5401* (2014).
- [11] Michael Hahn. “Theoretical Limitations of Self-Attention in Neural Sequence Models”. In: *Transactions of the Association for Computational Linguistics* (2020).
- [12] Yiding Hao et al. “Formal Language Recognition by Hard Attention Transformers: Perspectives from Circuit Complexity”. In: 2022.
- [13] Ari Holtzman et al. “The Curious Case of Neural Text Degeneration”. In: *International Conference on Learning Representations (ICLR)*. 2020.

- [14] Jared Kaplan et al. “Scaling Laws for Neural Language Models”. In: *arXiv preprint arXiv:2001.08361* (2020).
- [15] Jack Lindsey et al. “On the Biology of a Large Language Model”. In: *Transformer Circuits Thread* (2025). URL: <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- [16] Wang Ling et al. “Finding Function in Form: Compositional Character Models for Open Vocabulary Word Representation”. In: *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2015.
- [17] Yin Liu et al. “Learning Natural Language Inference Using Bidirectional LSTM Model and Inner-Attention”. In: *arXiv preprint arXiv:1605.09090* (2016).
- [18] William Merrill et al. “The Expressive Power of Transformers with Chain of Thought”. In: (2025).
- [19] Muhammad Ahmed Mohsin et al. *On the Fundamental Limits of LLMs at Scale*. 2026. arXiv: 2511.12869 [cs.LG]. URL: <https://arxiv.org/abs/2511.12869>.
- [20] Binghui Peng, Sridhar Narayanan, and Christos Papadimitriou. “On limitations of the transformer architecture”. In: *First conference on language modeling*. 2024.
- [21] Lena Strobl et al. “Formal Language Theory Meets Modern NLP: A Survey on Expressivity”. In: (2024).
- [22] Sainbayar Sukhbaatar et al. “End-to-End Memory Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015.
- [23] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.
- [24] Jason Weston, Sumit Chopra, and Antoine Bordes. “Memory Networks”. In: *International Conference on Learning Representations (ICLR)*. 2015.
- [25] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. *Hallucination is Inevitable: An Innate Limitation of Large Language Models*. 2025. arXiv: 2401.11817 [cs.CL]. URL: <https://arxiv.org/abs/2401.11817>.